

GAMEPLAN OLYMPUS CLIMB

Fase 1 – Setup & Configuração Inicial

Definição: Configurar ambiente de desenvolvimento local com todas as tecnologias e estruturas iniciais. Garantir ambiente estável antes de avançar.

DoD (Definition of Done) da Fase 1

- Ambiente local 100% funcional.
 - Infra Docker pronta.
 - Git inicial configurado com branches padrão.
 - Ambiente Expo funcional com Nativewind configurado.
 - Estrutura inicial NestJS com Prisma e banco PostgreSQL configurados.
 - Redis e BullMQ rodando e integrados.
-

Passo 1.1 – Git, Repositório & Estrutura Inicial

Definição: Criar e configurar repositório e estrutura inicial.

DoD

- Repositório criado no GitHub com branches main, develop e scraper-core-dev.
- Gitignore e editorconfig configurados.
- Estrutura básica de pastas criada.

Checklist

- Criar repo GitHub (`olympus-climb`)
- Branches (`main`, `develop`, `scraper-core-dev`)
- Criar `.gitignore`

- Criar `.editorconfig`
 - Inicializar repo local
-

Passo 1.2 – Docker Compose (Postgres, Redis, PgAdmin)

Definição: Ambiente dockerizado localmente estável e replicável.

DoD

- Containers PostgreSQL, Redis, PgAdmin rodando localmente.
- Dados persistentes via volume.

Checklist

- Criar `docker-compose.yml`
 - PostgreSQL container com persistência.
 - Redis container com persistência.
 - PgAdmin acessível e conectado automaticamente.
-

Passo 1.3 – Backend NestJS + Prisma

Definição: Estrutura inicial backend.

DoD

- Projeto NestJS funcional com Fastify.
- Prisma conectado a PostgreSQL.
- Migration inicial criada e Prisma Client gerado.

Checklist

- Instalar Nest CLI (`npm i -g @nestjs/cli`)

- Inicializar projeto NestJS Fastify
 - Prisma configurado (`npx prisma init`)
 - Migration inicial Prisma (`npx prisma migrate dev`)
 - Prisma Client (`npx prisma generate`)
 - Healthcheck básico (GET `/health`)
-

Passo 1.4 – Expo + React Native + Nativewind

Definição: Estrutura inicial frontend mobile.

DoD

- Expo rodando projeto inicial.
- Nativewind configurado e funcionando.
- Tokens de estilo básicos configurados.

Checklist

- Inicializar projeto Expo (`expo init`)
 - Configurar Nativewind
 - Criar tokens (`tailwind.config.js`)
 - Rodar projeto localmente sem erros (`expo start`)
-

Passo 1.5 – Redis + BullMQ Setup

Definição: Estrutura de filas BullMQ rodando.

DoD

- Redis rodando em Docker.

- Worker básico com BullMQ funcionando.

Checklist

- Instalar BullMQ
 - Worker teste criado (`src/workers/sample.worker.ts`)
 - Conexão Redis validada
-

Fase 2 – Arquitetura Modular & Componentização

Definição: Implementar arquitetura modular, desacoplada e escalável, aplicando DDD e CQRS no backend e Atomic Design no frontend.

DoD da Fase 2

- Estrutura backend organizada por domínios.
 - CQRS completo no backend.
 - Componentização frontend organizada (Atomic Design).
-

Passo 2.1 – Backend Modular (NestJS DDD + CQRS)

DoD

- Domínios: `Vaga`, `Empresa`, `ScrapingTask`
- Estrutura CQRS implementada (Commands, Queries, Handlers)
- Repository pattern com Prisma e InMemory

Checklist

- Criar módulos NestJS para domínios
- Implementar CQRS NestJS

- Estrutura com Commands e Queries separados
 - Repositórios InMemory e Prisma inicializados
-

Passo 2.2 – Frontend Atomic Design & Tokens

DoD

- Tokens centralizados funcionando
- Componentes `Atom`, `Molecule`, `Organism`

Checklist

- Criar estrutura Atomic (Atoms, Molecules, Organisms)
 - Tokens `designTokens.ts` validados
 - Componentes básicos (`Button`, `Card`, `Badge`)
-

Fase 3 – Implementação do Scraper Robustecido (BullMQ + Puppeteer)

Definição: Scraper rodando em fila BullMQ, estável, escalável e modularizado por fonte.

DoD da Fase 3

- Scraper rodando via fila (BullMQ worker)
 - Dados persistindo corretamente
 - Estrutura para múltiplas fontes de scraping criada
-

Passo 3.1 – Workers BullMQ

DoD

- Worker rodando scraping básico
- Fila robusta com retry, delay configurados

Checklist

- Implementar Workers BullMQ ([src/workers/jobScraper.worker.ts](#))
 - Configuração retry, backoff, delay
-

Passo 3.2 – Puppeteer Scraping Modules

DoD

- Scraper funcional com Puppeteer
- Modularização pronta (ex: LinkedInScraper, GupyScraper)

Checklist

- Implementar Puppeteer básico
 - Scrapers separados por fontes validados
 - Parsing estável de HTML pra JSON validado
-

Fase 4 – Dashboard, Métricas e Filtros Avançados

Definição: Backend com filtros avançados, frontend dashboard com métricas robustas.

DoD da Fase 4

- Dashboard backend com filtros complexos validados
 - Frontend mostrando métricas e filtros corretamente
-

Passo 4.1 – Backend Dashboard & Filtros Prisma

DoD

- Filtros (AND/OR) implementados via Prisma
- Métricas (count, sum, groupBy) validadas

Checklist

- Criar queries Prisma com filtros (igual SOLUTIS_SERASA)
 - Testes unitários e integração nos filtros
-

Passo 4.2 – Frontend Dashboard

DoD

- Dashboard mobile mostrando métricas e filtros
- Componentização avançada aplicada (Atomic)

Checklist

- Componentes Dashboard ([JobCard](#), [MetricsCard](#))
 - Filtros frontend ([SearchBar](#), [Dropdown](#))
-

Fase 5 – Testes (Unitários, Integração, E2E) & Coverage Completo

Definição: Implementar testes abrangentes e robustos para garantir estabilidade contínua.

DoD da Fase 5

- Coverage mínimo de 90%
 - Relatório Jest HTML gerado automaticamente
-

Passo 5.1 – Testes Unitários

Checklist

- Criar testes unitários (Jest) para todos módulos
- Validar coverage

Passo 5.2 – Testes de Integração e E2E

Checklist

- Testes com Prisma real mockado
 - Testes E2E com Artillery validados
-

Fase 6 – CI/CD Automatizado & Observabilidade

Definição: Garantir integração e deploy contínuo com monitoramento robusto.

DoD da Fase 6

- GitHub Actions rodando com sucesso
 - PM2 gerenciando processos em produção
 - Logging estruturado funcionando
-

Passo 6.1 – GitHub Actions CI/CD

Checklist

- Configurar CI/CD ([.github/workflows/](#))
- Validar actions (lint, tests, build)

Passo 6.2 – PM2 e Logging

Checklist

- PM2 cluster mode configurado
 - Logging estruturado com Pino validado
-

Fase 7 – Deploy Público & OTA Expo

Definição: Aplicação estável e pública com atualização OTA no frontend.

DoD da Fase 7

- Backend público rodando em Render
 - Frontend publicado com EAS OTA
-

Passo 7.1 – Deploy Backend

Checklist

- Configurar e subir backend no Render
- Validar Healthcheck público

Passo 7.2 – Deploy Expo OTA

Checklist

- Deploy Expo via EAS Build
 - Configuração EAS Update validada
-

Resultado Final Esperado:

- Ambiente profissional robusto
- Cobertura de testes acima de 90%
- Documentação detalhada (README e TypeDoc)

- Escalabilidade garantida via PM2, BullMQ, Docker
- Performance validada com Artillery